1. Context compression

This one is **much more commercially attractive than it initially looks**.

Imagine you build:

> **ContextOS — the system that tells an AI exactly what information it can actually use reliably.**

Instead of:

```
Model supports 1,000,000 tokens
```

your system says:

```
Effective context:

0–20k       97% retrieval/use
20–80k      91%
80–200k     73%
200–500k    41%
500k–1M     18%

Task: legal-document reasoning

Safe operating range: 120k tokens
```

That's valuable.

Enterprise AI already has a major **context problem**, with organizations struggling with fragmented, stale or inconsistent context feeding agents.

The startup

You could sell:

- context benchmarking
- context-quality scoring
- effective-context measurement
- automatic context compression
- intelligent retrieval
- context routing
- context guarantees
- model/context optimization

Eventually:

> **"Datadog for AI context."**

Your path

Phase 1 — Research

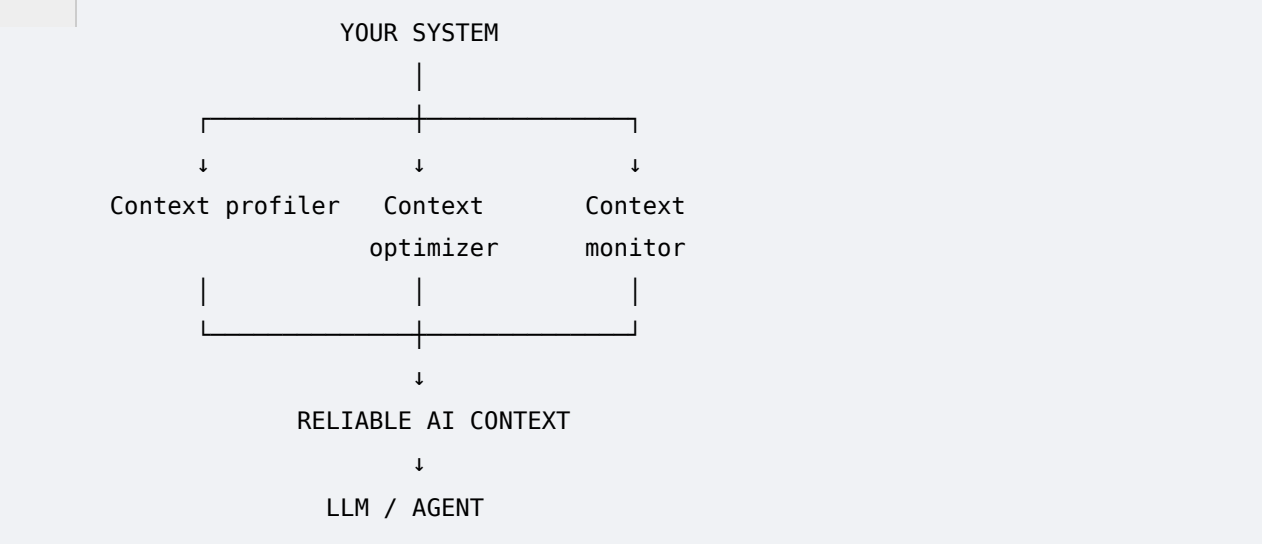
- Define exactly what “effective context” means.
- Measure how performance changes with token position, context length, task type, and model.
- Determine whether there is a predictable law.
- Identify the mechanisms: positional effects, attention dilution, retrieval interference, etc.
- Test whether your theory predicts failure *before inference*.

Phase 2 — Breakthrough

- Develop a predictive **Effective Context Law**.
- Create a benchmark that measures effective context rather than nominal context window.
- Develop your proposed positional curriculum / sparse-hierarchical attention method.
- Show that it improves the measurable limit.

Phase 3 — Product

Build an API/platform like:



A company could pay you to answer:

> **“How much of my model's 1M-token context can I actually trust for this task, and how should I construct the context to maximize accuracy?”**

That is a much stronger commercial proposition than simply saying **“we compress context.”**

And your first milestone should NOT be a startup.

Your first target should be:

> **Can we experimentally demonstrate a previously unknown, predictable law governing effective context?**

If you can establish that convincingly, **then** we build the product around it.

For **Option 1**, I'd build toward one concrete objective:

> **Create a predictive "effective context" model that tells you how much of an LLM's advertised context is actually usable for a specific task, then build a runtime system that automatically optimizes context before the model sees it.**

The opportunity is real, but the research question needs sharpening. Existing work already shows that nominal context length can badly overstate practical performance: *Lost in the Middle* found strong position-dependent degradation, RULER showed many models fail well before their advertised context size as task complexity increases, and BABILong reported that models can effectively use only a fraction of very long contexts on reasoning tasks. Anthropic also explicitly describes context as a finite resource with diminishing usefulness as prompts grow.

So your novelty cannot simply be **"long contexts degrade."** That is already known.

Your target should be:

> **Predict the degradation curve before running the task, generalize it across models/tasks, then modify context/training so the predicted usable-context boundary moves outward.**

Stage 0 — Define exactly what you are discovering

Before coding, define a quantity.

Call it **Effective Context Capacity**, ECC.

Suppose a model advertises:

$C_{\max}=128{,}000$

tokens.

That does **not** mean all 128k tokens are equally usable.

Define something like:

$$ECC(M,T,\tau)=\max\{L : P(M,T,L)\ge \tau P(M,T,L_0)\}$$

where:

- M = model
- T = task
- L = context length
- L_0 = short-context baseline
- P = task performance
- τ = acceptable retained-performance threshold

For example, if short-context accuracy is 90%, and you define $\tau=0.9$:

$\text{threshold}=0.9(90\%)=81\%$

Perhaps performance looks like:

Length	Accuracy
4k	90%
8k	89%
16k	88%
32k	85%
64k	77%
128k	62%

Then ECC under that particular definition is somewhere between **32k and 64k**.

That's already more meaningful than:

> Model supports 128k.

But eventually we need something much richer than one number.

Stage 1 — First 2 weeks: reproduce known results

Do **not** invent your theory immediately.

First prove you can reproduce the phenomenon.

Start with three families of experiments.

Experiment A — Position

Put one required piece of information at:

$p \in \{0.05,0.10,0.25,0.50,0.75,0.90,0.95\}$

of the prompt.

Measure whether the model retrieves/uses it.

This recreates the basic phenomenon studied by *Lost in the Middle*.

Experiment B — Length

Keep the task identical.

Change only:

L=2k,4k,8k,16k,32k,64k,…L= 2k,4k,8k,16k,32k,64k,\ldots

Then measure:

Accuracy=f(L)Accuracy=f(L)

RULER is particularly useful here because it was explicitly designed to vary context length and task complexity rather than relying only on simple needle retrieval.

Experiment C — Complexity

Now vary how much information must be combined.

For example:

Level 1

> Retrieve one fact.

Level 2

> Retrieve two facts.

Level 3

> Combine A + B.

Level 4

> Follow A → B → C → D.

Level 5

> Aggregate many facts.

BABILong is useful inspiration because it tests reasoning when necessary facts are distributed through long irrelevant contexts.

Stage 2 — Build your benchmark

Weeks **3–6**.

Don't depend entirely on existing benchmarks.

Create your own controllable generator.

Call it something temporary like:

ContextLab

It should generate prompts where you know exactly:

length\text{length} position\text{position} number of relevant facts\text{number of relevant facts}
distance between relevant facts\text{distance between relevant facts}
distractor similarity\text{distractor similarity} reasoning depth\text{reasoning depth}
answer\text{answer}

This controllability is extremely important.

You want something like:

```
sample = generate_task(
    length=32000,
    relevant_positions=[0.12, 0.63, 0.87],
    reasoning_hops=3,
    distractor_similarity=0.7,
    task="causal_chain"
)
```

And automatically receive:

```
{
    "prompt": "...",
    "answer": "K17",
    "metadata": {
        "tokens": 31984,
        "positions": [...],
        "reasoning_depth": 3
    }
}
```

Now experiments become cheap.

Stage 3 — Create the Context Performance Surface

Month **2**.

This is where your work starts becoming interesting.

Don't study only:

Performance=f(ContextLength)Performance=f(ContextLength)

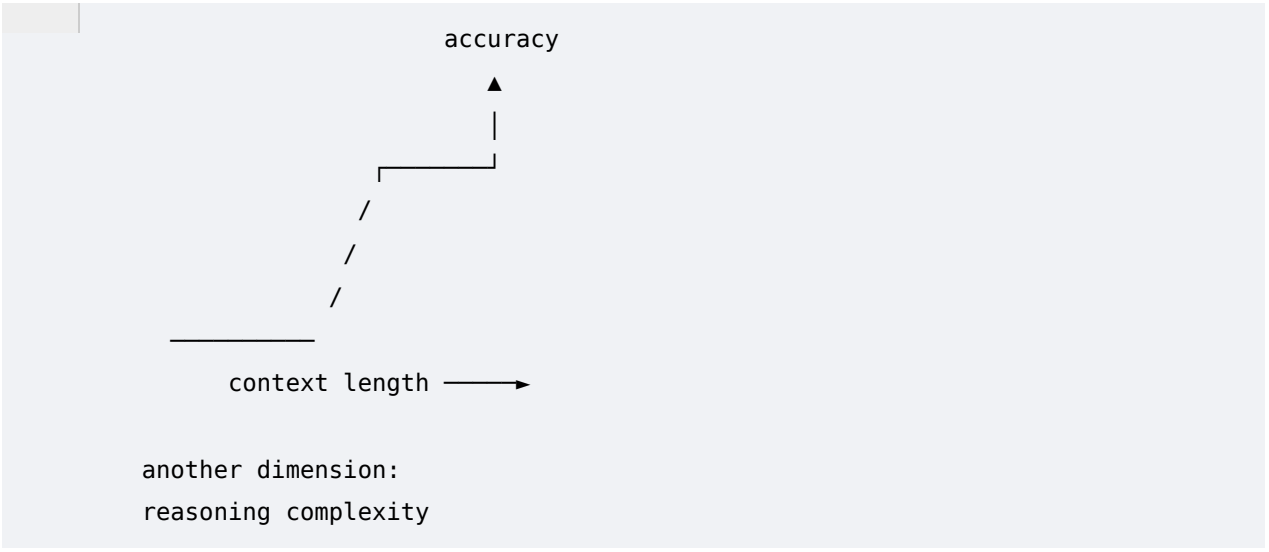
Study:

P=f(L,p,d,r,n,s,M)P = f(L, p, d, r, n, s, M)

where perhaps:

- LL = context length
- pp = relevant-information position
- dd = distance between dependent facts
- rr = reasoning depth
- nn = number of required facts
- ss = distractor similarity
- MM = model

Now imagine plotting:



You aren't finding a single context number anymore.

You're discovering a **context capability surface**.

Stage 4 — Search for a law

Months **2–4**.

Now ask:

> Can degradation across different models collapse onto a common mathematical form?

For example, maybe something approximately resembles:

$P(L)=P0e−αLP(L)=P_0e^{-\alpha L}$

Probably too simple.

Maybe:

$P=P0exp[−α(LC)β]P= P_0 \exp \left[-\alpha \left(\frac{L}{C}\right)^\beta \right]$

Now introduce task complexity:

$P=P0exp[−α(LRC)β]P= P_0 \exp \left[-\alpha \left(\frac{L R}{C} \right)^\beta \right]$

where RR represents some effective reasoning burden.

Or perhaps the right variable is not raw context length at all.

Maybe it's:

$\frac{\text{distractor entropy}\times\text{dependency distance}\times\text{reasoning depth}}{\text{model capacity}}$

That's exactly why you're doing research.

You don't assume the equation.

You discover it.

Stage 5 — The breakthrough criterion

This is critical.

Finding a curve that fits existing measurements isn't enough.

You want this experiment:

Train/fit your law using:

- model A
- model B
- model C
- contexts up to 32k
- tasks 1–4

Then hide:

- model D
- 64k/128k contexts
- task 5

Your theory predicts:

$Accuracy_{predicted}=63\%$

Then experiments produce:

$Accuracy_{observed}=65\%$

That's interesting.

Do it repeatedly.

If the law generalizes out-of-distribution, then you may actually have something significant.

Stage 6 — Investigate the mechanism

Months **3–6**.

Now move from:

> Here's what happens.

to:

> Here's why it happens.

Study candidate mechanisms separately.

For example:

Positional degradation

Compare identical information appearing early/middle/late.

Attention competition

Increase the number of semantically similar distractors.

Dependency distance

Compare facts separated by:

100, 1k, 10k, 50k100,\ 1k,\ 10k,\ 50k

tokens.

Number of relevant items

Compare:

1,2,4,8,16,321,2,4,8,16,32

necessary pieces of evidence.

Reasoning depth

Compare:

1,2,3,4,5,81,2,3,4,5,8

logical hops.

Context composition

Compare:

```
random text
```

versus:

```
semantically unrelated prose
```

versus:

```
highly similar distractors
```

This distinction matters enormously.

64k random tokens and 64k highly plausible distractors aren't necessarily the same information-processing problem.

Stage 7 — Test multiple models

Don't build your theory around one API.

Eventually test at least three categories:

Small open models

Useful because you can inspect internals.

For example, models in the roughly:

1B−8B1B-8B

parameter range.

Larger open models

Useful for checking scaling behavior.

Closed frontier models

Useful for checking whether the theory generalizes to production systems.

Your result becomes much stronger if:

ECC=f(architecture,size,training,task)ECC=f(\text{architecture},\text{size},\text{training},\text{task})

rather than:

> I discovered a strange behavior in one Llama checkpoint.

Stage 8 — Your first paper

Around months **4–7**, depending on results.

A hypothetical title:

> **Beyond Nominal Context: Predicting the Effective Context Capacity of Large Language Models**

Possible contributions:

1. Formal definition of effective context.
2. Controllable context-stress benchmark.
3. Large empirical study.
4. Predictive degradation model.
5. Cross-model generalization.
6. Mechanistic analysis.

Do **not** force a startup yet.

At this stage, success is:

> Other researchers start using your metric.

That's incredibly valuable.

Stage 9 — Now attack the limit

Once you've understood the boundary, try moving it.

This is your second research project.

You originally mentioned two ideas.

A. Positional curriculum

During training/fine-tuning, deliberately vary where critical information appears.

Instead of critical information usually occurring near predictable positions:

```
training example 1: critical @ 5%
training example 2: critical @ 51%
training example 3: critical @ 93%
training example 4: critical @ 27%
```

and deliberately vary dependency distance.

Test:

ECCafter>ECCbeforeECC_{after}>ECC_{before}

while keeping:

- model size constant
- inference compute roughly constant
- task difficulty constant

That would be meaningful.

Stage 10 — Try architectural improvements

More ambitious.

You mentioned sparse/hierarchical attention.

A conceptual system might operate like:

```
128k context
       ↓
local representation
       ↓
segment summaries
       ↓
important-state routing
       ↓
global hierarchy
       ↓
reasoning
```

Instead of every token being treated uniformly.

But don't begin here.

Architectural research is expensive and introduces too many variables.

Measure first.

Then fix.

Stage 11 — Turn the benchmark into open source

Months **6–9**.

Build something developers can run:

```
pip install contextlab
```

Then:

```
contextlab test \
    --model model_name \
    --task rag \
    --max-context 128000
```

Output:

```
Model: X
Advertised Context: 128K

Effective Context
-----------------
Retrieval:        103K
Multi-hop QA:      57K
Dense reasoning:   31K
Agent history:     42K

Middle-position penalty: 17%
Distractor sensitivity: HIGH
Dependency-distance sensitivity: MEDIUM
```

Now we're crossing from research into product.

Stage 12 — Build the killer visualization

Imagine a developer dashboard.

```
Model X — Context Reliability

100% ┤████████
 90% ┤███████
 80% ┤██████
 70% ┤█████
 60% ┤████
 50% ┤███
     └────────────────────
```

```
           4  8  16  32  64  128K
                  tokens
```

Then:

```
Task                    Safe Context

Simple retrieval             105K
Multi-hop retrieval           68K
Document QA                   61K
Code reasoning                42K
Agent history                 37K
```

That becomes immediately understandable to engineering teams.

Stage 13 — Your startup's first actual product

The first commercial product should **not** require retraining models.

That's important.

Sell something easy to adopt.

Build:

Context Profiler

Company connects its agent/RAG pipeline.

You run experiments automatically.

Then report:

```
Your agent currently sends:
86,400 tokens/request

Effective usable context:
~41,000 tokens

Estimated wasted context:
45,400 tokens

Observed failure risk:
High after 54K

Recommendation:
retrieve 12 documents instead of 30
move policy instructions to ...
```

```
compress conversation history ...
retain evidence clusters ...
```

That's already something companies could pay for.

Stage 14 — Context optimizer

Now don't just diagnose.

Automatically fix it.

Pipeline:

```
RAW CONTEXT
     ↓
Context Analyzer
     ↓
Importance estimator
     ↓
Dependency detector
     ↓
Context budget allocator
     ↓
Ordering optimizer
     ↓
Compression / retrieval
     ↓
OPTIMIZED CONTEXT
     ↓
LLM
```

The application sends:

```
optimized = context_engine.optimize(
    model="...",
    task="legal_qa",
    context=documents
)
```

And your system selects:

- what stays
- what goes
- what gets summarized
- what gets retrieved
- where evidence is placed

- how much context to use

Stage 15 — Your most important commercial metric

Eventually quantify something businesses understand:

Accuracy per input dollar\text{Accuracy per input dollar}

or:

Useful information per token\text{Useful information per token}

Suppose customer currently gets:

```
Accuracy: 83%
Input: 90K tokens
Cost: X
```

Your system:

```
Accuracy: 91%
Input: 34K tokens
Cost: 0.4X
```

Now you're not selling academic theory.

You're selling:

> **better reliability + lower inference cost.**

That's much easier to monetize.

Stage 16 — First customers

Don't initially sell to banks.

Too slow.

Find AI startups with:

- RAG systems
- coding agents
- legal-document analysis
- customer-support agents
- research agents
- document intelligence
- long-running agents

Anthropic itself describes context management as an important problem for long-running agents, including situations where compaction alone is insufficient.

Your first pitch becomes:

> Give me 500 representative agent runs. We'll measure exactly where your context begins hurting performance and whether you're paying for tokens your model isn't effectively using.

Much stronger than:

> We've invented a theoretical long-context metric.

Stage 17 — Pricing

Early stage:

```
Free
benchmark one model
```

Developer:

```
$50–$500/month
```

Startup:

```
$1k–$5k/month
```

Enterprise:

```
$25k–$250k+/year
```

Eventually usage-based optimization could be:

```
$X / million optimized tokens
```

or based on inference savings.

Those aren't guarantees; they're sensible pricing experiments once there is measurable customer ROI.

Stage 18 — Moat

This is important because context optimization itself will become crowded.

Your moat should not be:

> We summarize prompts.

Anyone can do that.

Your moat should become four things:

1. Proprietary measurement dataset

Millions/billions of observations:

(model,task,length,position,structure)→performance(model,task,length,position,structure)

\rightarrow performance

2. Predictive context model

You know beforehand:

P(success∣context,model,task)P(success\mid context,model,task)

3. Runtime optimizer

Automatically restructures context.

4. Continuous feedback

Every customer's workload makes the predictor better.

That creates a flywheel:

```
more customers
     ↓
more workload data
     ↓
better context model
     ↓
better optimization
     ↓
better customer results
     ↓
more customers
```

Stage 19 — Larger vision

At this stage, stop thinking:

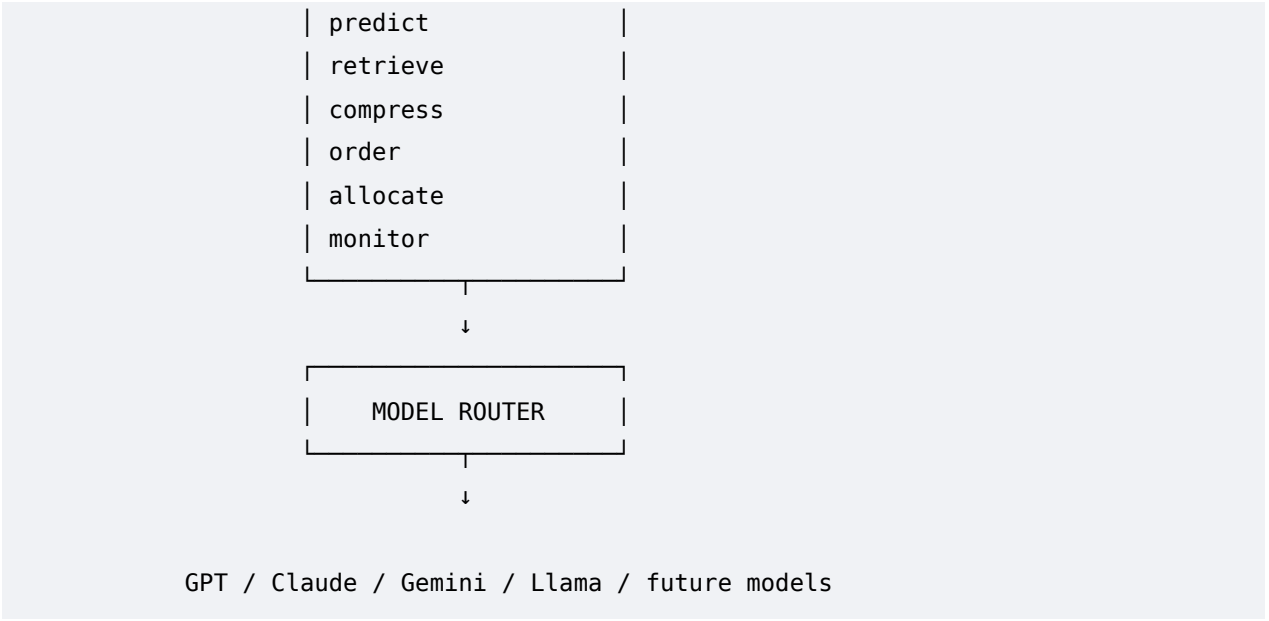
> Context compression company.

The larger idea is:

Context Intelligence Layer

Something sitting between applications and models.

```
                    APPLICATION
                         │
                         ↓
                ┌──────────────────────┐
                │  CONTEXT INTELLIGENCE│
                │                      │
                │ measure              │
```



You become model-agnostic infrastructure.